CSE6250: Big Data Analytics in Healthcare
Project Final Report

LLMSYN – Generating Synthetic Electronic Health Records Without Patient-Level Data

Team Number: 22 Team ID: E2
Sahil Singhal **Goyeun Yun**

November 2025

Abstract

This project reproduces the LLMSYN framework for generating synthetic Electronic Health Records (EHRs) without patient-level data. We reconstruct the full four-step LLM-based pipeline (demographics → primary diagnosis → comorbidities → procedures) and systematically evaluate three LLMSYN variants using the MIMIC-III dataset. Our reproduction assesses utility (downstream performance on respiratory phenotyping and mortality prediction) and fidelity (marginal and joint distribution similarity using KS and MMD metrics).

Across 100-patient synthetic cohorts, we find that: (1) data augmentation maintains predictive performance comparable to real-only models, suggesting that synthetic data does not harm downstream learning, (2) fully synthetic models achieve competitive but less stable performance, and (3) demographic fidelity is strong, while diagnosis- and comorbidity-level fidelity remains limited. Key discrepancies from the original LLMSYN paper arise from our smaller synthetic cohort size, limited ICD-9 coverage, and unavailable prompt templates and grounding sources.

Overall, LLMSYN is partially reproducible: its qualitative findings are validated, but exact numerical replication is limited by dataset scale and missing implementation details.

Project Video: Click to Watch
GitHub Repository: Go to Repo

1 Introduction

Electronic Health Records (EHRs) are essential for developing predictive models in healthcare applications [11, 15, 17]. However, accessing real patient-level data poses significant challenges due to privacy risks and regulatory restrictions such as HIPAA in the United States and GDPR in Europe. Prior work has demonstrated that even de-identified data can still be vulnerable to re-identification attacks [13, 3, 6]. Synthetic data generation has emerged as a promising solution to balance privacy and utility. Traditional generative models such as GANs and diffusion models require substantial real patient data for training and may still leak sensitive information. Rule-based simulators such as Synthea [2] avoid this issue but often fail to produce clinically useful patterns.

Hao et al. [8] introduce LLMSYN, a novel synthetic EHR generation pipeline that uses large language models (LLMs) without requiring any patient-level training data. LLMSYN applies a multi-step Markov-style prompting framework to sequentially generate demographics, diagnoses, complications, and procedures while grounding each step using external medical knowledge and aggregate statistics. This approach demonstrates competitive utility, strong fidelity, and minimal privacy risk, highlighting the potential for LLM-based synthetic data generation.

In this reproduction study, we aim to verify LLMSYN's main claims using the MIMIC-III dataset and independently implemented generation and evaluation pipelines.

2 Scope of Reproducibility

The LLMSYN paper proposes that large language models can generate clinically meaningful synthetic EHR data without requiring access to patient-level records. In this reproducibility study, we aim to replicate two major components of the original work: (1) the synthetic data generation pipeline itself (LLMSYN-base, LLMSYN-prior, LLMSYN-full), and (2) the empirical claims reported in the paper.

Specifically, we focus on evaluating two central claims made by the authors: **Utility** (synthetic EHRs can support downstream predictive modeling with performance comparable to real data) and **Fidelity** (synthetic samples preserve the statistical characteristics of real clinical distributions). The privacy claim made in the original work is not reproduced due to dataset size limitations (100 synthetic samples per variant). Table 1 summarizes the hypotheses and the experimental setups designed to test each claim.

3 Methodology

3.1 Dataset and Synthetic Variants

We use the publicly available MIMIC-III v1.4 ICU database[1], which contains de-identified EHRs with demographics, ICD-9 diagnoses/procedures, and in-hospital mortality. Following LLMSYN, we build a cohort of adult, non-newborn admissions and derive patient-level tables with demographics, a primary ICD-9 diagnosis, comorbid ICD-9 codes, procedures, and the binary mortality label `HOSPITAL_EXPIRE_FLAG`.

For synthetic data, we reproduce three LLMSYN variants, each with 100 synthetic patients:

`LLMSYN_base` (no priors or external knowledge), `LLMSYN_prior` (MIMIC-derived demographic

[1] `https://physionet.org/content/mimiciii/1.4/`

Table 1: Summary of Reproducibility Hypotheses and Evaluation Plan

Hypothesis	Claim	Evaluation Method	Success Criteria
Utility	Synthetic EHRs generated by LLMSYN achieve performance comparable to real data on downstream predictive tasks	Train ML models under REAL, FS, and DA regimes and compare Accuracy and AUROC.	FS and DA within 5–10% of REAL and follow original paper trends.
Fidelity	LLMSYN produces synthetic distributions similar to real EHR data.	Compute KS (marginal) and MMD (joint) across different LLMSYN variants	Ordering matches original paper: full $>$ prior $>$ base.

and ICD-9 frequency priors), and `LLMSYN_full` (priors + Mayo Clinic disease descriptions + prompt adaptor). Priors include age-bin distributions, sex ratio, language/insurance mixtures, mortality rate, and top-100 ICD-9 counts. All priors are computed from aggregate statistics only.

3.2 LLMSYN Pipeline

We implement a four-step, prompt-based generation pipeline:

1. **Demographics:** Generate AGE, GENDER, LANGUAGE, RELIGION, MARITAL_STATUS, ETHNICITY, INSURANCE, and MORTALITY flag, conditioned on aggregate priors.
2. **Main Diagnosis:** Sample a single ICD-9 code using the demographic profile and the top-100 ICD-9 prior table.
3. **Comorbidities:** Generate a short list of additional ICD-9 codes given demographics, main diagnosis, and (for prior/full) Mayo Clinic disease descriptions.
4. **Procedures:** Generate ICD-9 procedure codes consistent with the diagnosis and comorbidity set, using a simple procedure catalogue.

Each stage consumes the previous stage's output, forming a Markov-style conditional chain. We used GPT 5.1 (Thinking) for all generations. Full prompt templates and example logs are included in the Appendix and Github repository.

3.3 Downstream Prediction with Synthetic Patient Data

We evaluate synthetic data on two tasks:

Respiratory phenotyping: Binary label indicating whether the main ICD-9 code falls in the respiratory chapter (460–519).

Mortality prediction: Binary in-hospital mortality from `HOSPITAL_EXPIRE_FLAG`.

For each patient, we construct simple, tabular features: AGE, categorical encodings of LANGUAGE, ETHNICITY, INSURANCE, RELIGION, MARITAL_STATUS, and the length of the comorbidity list (`OTHER_ICD_COUNT`). Diagnosis and procedure codes are used only for labels and fidelity metrics, not as direct features.

At each random seed, we sample 100 real patients for training and 100 distinct real patients for testing. Depending on the regime, the training set is REAL(100), SYN(100), or REAL(100)+SYN(100);

the test set is always REAL(100). All experiments are repeated for 10 seeds.

3.4 Models, Metrics, and Implementation

For utility, we train a RandomForestClassifier (scikit-learn) with $n\_estimators$=200 and default settings otherwise (n_jobs= $-$ 1). We report Accuracy and AUROC averaged over 10 seeds. For fidelity, we compute KS statistics for marginal distributions (demographics, AGE, comorbidity/procedure counts) and MMD^2 with an RBF kernel for joint code-pairs (MAIN+PROC, MAIN+COMORB, COMORB+PROC).

All code (preprocessing, priors, generation wrappers, and evaluation scripts) was written in Python. LLMs were used as a coding assistant (e.g., for template generation and debugging), but we manually reviewed and adapted all code before use.

4 Training

4.1 Computational Implementation

LLMSYN itself does not involve training a neural network; instead, it relies on LLM inference to generate synthetic EHR samples, followed by training conventional machine learning models on either real or synthetic data for downstream evaluation.

Hardware: All experiments were conducted on a commodity computing environment:

- CPU: Apple M4
- RAM: 16 GB
- GPU: Not used for training; all models were trained on CPU.
- LLM access: GPT 5.1 (Thinking) via API

LLM Generation Runtime: For each synthetic batch, we generate 100 patient records using the 4-step LLMSYN pipeline (demographics, main diagnosis, complications, procedures). On average:

- Number of prompts per patient: 4–6 (including retries).
- Average latency per prompt: 57 seconds.
- Wall-clock time per 100-patient batch: approximately 4 minutes.
- Total number of synthetic batches generated: 3 batches (full, prior, and base)

5 Evaluation

We evaluate LLMSYN along two dimensions consistent with the original paper: **(1) Utility** and **(2) Fidelity**. All metric code and evaluation scripts were drafted with the help of an LLM (see Appendix for generated code).

5.1 Utility Evaluation (Predictive Performance)

Task: Binary respiratory phenotype prediction: MAIN_ICD9 $\in [460, 519]$ is labeled 1, else 0. (Used only as labels; diagnosis/procedure codes are not used as features.)

Features: AGE, five categorical groups (LANGUAGE, ETHNICITY, INSURANCE, RELIGION, MARITAL_STATUS), and a scalar comorbidity feature (OTHER_ICD_COUNT). Although the pipeline contains seven feature groups in total, the comorbidity signal is encoded as a single integer representing the length of the COMORBIDITIES list.

Datasets and Splits: For fair comparison with synthetic scale, all experiments use:

- REAL: 100 stratified samples per seed
- SYN: 100 synthetic samples per variant (base / prior / full)
- FS: SYN(100) $\rightarrow$ REAL(100)
- DA: REAL(100) + SYN(100) $\rightarrow$ REAL(100)
- REAL_match: REAL(100) $\rightarrow$ REAL(100) (non-overlapping when possible)

All settings are repeated for 10 random seeds.

Model: RandomForestClassifier ($n\_estimators$=200, n_jobs= $-$ 1).

Metrics: Accuracy and AUROC.

5.2 Fidelity Evaluation (Statistical Similarity)

To assess how closely synthetic data resembles real clinical distributions, we compute:

- **Kolmogorov–Smirnov (KS)** statistics for marginal distributions (AGE, comorbidity length, demographic categories).
- **Maximum Mean Discrepancy (MMD^2)** with RBF kernels to evaluate joint distribution (MAIN+PROC, MAIN+COMORB, COMORB+PROC) similarity across variants.

6 Results

In this section, we report empirical results from all LLMSYN variants (LLMSYN_base, LLMSYN_prior, LLMSYN_full) across two evaluation dimensions: (1) utility and (2) fidelity.

6.1 Utility Results

We evaluate LLMSYN-generated synthetic EHRs on two downstream tasks: (i) Respiratory phenotyping (binary classification: ICD9 460–519 vs. Other), and (ii) In-hospital mortality prediction. All experiments use 100 samples for training and 100 real samples for testing per seed, repeated for 10 seeds.

Table 2 reports the averaged Accuracy and AUROC. Overall, three key observations emerge:

- **Data Augmentation (DA, REAL + SYN)** maintains AUROC comparable to REAL-only models.
- **FS (Fully Synthetic) is competitive but less stable**, especially for mortality, reflecting small-sample variance and limited phenotype coverage.
- **LLMSYN_prior does not consistently outperform base/full**, likely due to synthetic cohort size (100) and noisy ICD9 distributions in our generated prior variant.

Comparison to Original LLMSYN Paper: Across both tasks, Accuracy remains consistently high ($\approx$ 0.95 for phenotyping and 0.83–0.89 for mortality), largely reflecting class imbalance. AUROC provides a more reliable comparison.

For respiratory phenotyping, FS and DA models perform comparably to the REAL baseline. Unlike the original LLMSYN paper, our results do *not* show a clear ordering of DA $>$ REAL $>$ FS. Notably, FS(syn_full) achieves the highest AUROC (0.624), suggesting that the LLM-generated feature distribution, despite low fidelity, still captures signal relevant to this binary task.

For mortality prediction, DA models show AUROC values that are comparable to REAL, indicating that synthetic augmentation does not degrade model performance. Small positive differences

Table 2: Utility Evaluation Summary (Phenotyping and Mortality)

Regime	Train	Test	ACC	AUROC
Phenotyping (Respiratory vs Other)				
DA (syn_base)	REAL(100)+SYN(100)	REAL(100)	0.960 ± 0.000	0.512 ± 0.153
DA (syn_full)	REAL(100)+SYN(100)	REAL(100)	0.960 ± 0.000	0.540 ± 0.184
DA (syn_prior)	REAL(100)+SYN(100)	REAL(100)	0.960 ± 0.000	0.534 ± 0.175
FS (syn_base)	SYN(100)	REAL(100)	0.960 ± 0.000	0.452 ± 0.177
FS (syn_full)	SYN(100)	REAL(100)	0.958 ± 0.004	0.624 ± 0.127
FS (syn_prior)	SYN(100)	REAL(100)	0.960 ± 0.000	0.499 ± 0.144
REAL_match	REAL(100)	REAL(100)	0.958 ± 0.004	0.528 ± 0.164
Mortality Prediction				
DA (syn_base)	REAL(100)+SYN(100)	REAL(100)	0.885 ± 0.008	0.590 ± 0.091
DA (syn_full)	REAL(100)+SYN(100)	REAL(100)	0.881 ± 0.007	0.607 ± 0.112
DA (syn_prior)	REAL(100)+SYN(100)	REAL(100)	0.882 ± 0.010	0.573 ± 0.114
FS (syn_base)	SYN(100)	REAL(100)	0.867 ± 0.013	0.506 ± 0.060
FS (syn_full)	SYN(100)	REAL(100)	0.831 ± 0.043	0.516 ± 0.079
FS (syn_prior)	SYN(100)	REAL(100)	0.866 ± 0.014	0.425 ± 0.108
REAL_match	REAL(100)	REAL(100)	0.877 ± 0.016	0.602 ± 0.108

were observed, but these fall within the sampling variance of the 100-sample regime. FS models show mixed performance: FS(syn_base) is competitive, while FS(syn_prior) underperforms due to unstable synthetic label distributions.

Differences from the original paper primarily stem from our restricted synthetic cohort size (100 samples), narrower ICD-9 phenotype coverage, and small-sample instability in random forest classifiers under outcome imbalance.

6.2 Fidelity Results

We evaluate marginal fidelity using Kolmogorov–Smirnov (KS) tests and joint fidelity using Maximum Mean Discrepancy (MMD). Lower values indicate closer agreement between synthetic and real distributions. Table 3 summarizes KS statistics across demographic and clinical features, and Table 4 reports MMD over key code-pair interactions.

Marginal Fidelity (KS): Demographic variables (LANGUAGE, RELIGION, MARITAL STATUS, ETHNICITY, INSURANCE, AGE) show consistently low KS distances for LLMSYN-full (typically 0.02–0.08) and LLMSYN-prior (0.04–0.08 range), indicating that both models successfully preserve population-level demographic structure. Mortality (HOSPITAL_EXPIRE_FLAG) is also captured reasonably well for all variants (KS is about 0.04–0.07).

In contrast, clinical structure shows substantially worse fidelity. MAIN_DIAGNOSIS exhibits very large divergence across all variants (KS 0.73–0.82), reflecting heavy-tailed ICD-9 distributions and limited coverage in the 100-sample synthetic cohort. Comorbidity count also diverges significantly (KS 0.57–0.91), consistent with LLMs under-generating secondary diagnoses. Procedure count fidelity is moderate, with LLMSYN-full achieving the best alignment (KS = 0.32).

Overall, LLMSYN-full provides the strongest demographic fidelity, while diagnosis-level and co-morbidity fidelity remain limited across all model variants.

Table 3: KS statistics comparing synthetic vs. real marginal distributions (lower = better).

Feature	Full	Prior	Base	Feature	Full	Prior	Base
LANGUAGE	0.080	0.060	0.330	HOSP_EXPIRE_FLAG	**0.044**	0.074	**0.044**
RELIGION	0.056	0.056	0.239	AGE	**0.077**	0.077	0.337
MARITAL_STATUS	**0.022**	0.047	0.352	MAIN_DIAGNOSIS	**0.731**	0.788	0.823
ETHNICITY	**0.052**	0.064	0.511	OTHER_ICD_COUNT	**0.572**	0.907	0.783
INSURANCE	0.023	0.083	0.427	PROCEDURE_COUNT	**0.320**	0.395	0.395

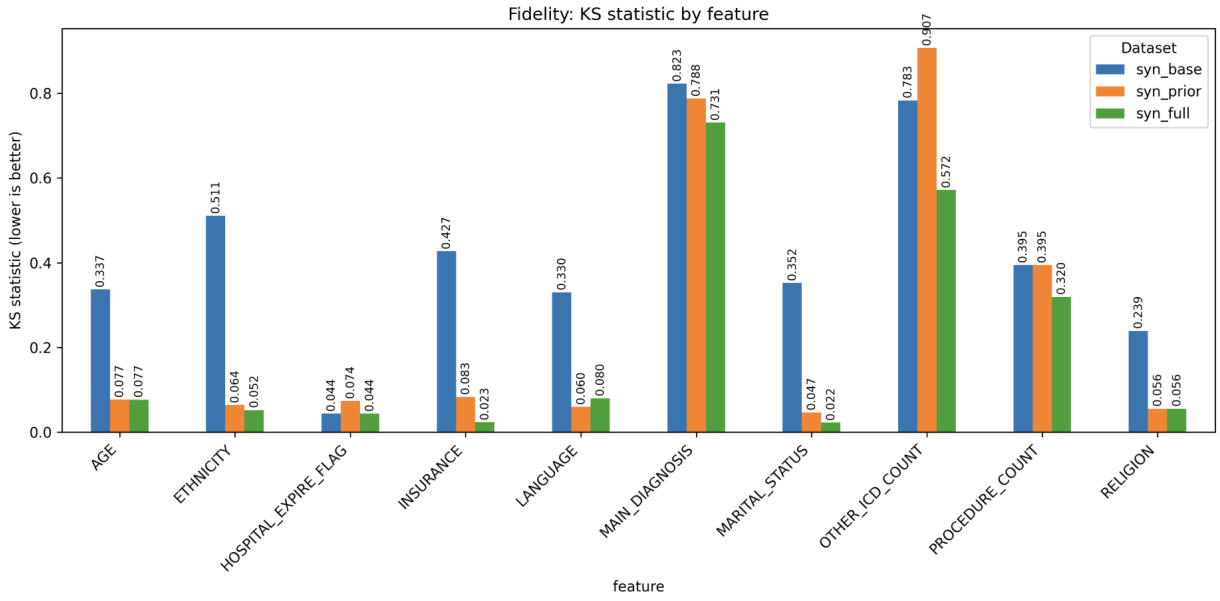

Figure 1: KS Statistic by Feature

Joint Fidelity (MMD): Table 4 shows that LLMSYN-base and LLMSYN-prior consistently achieve lower MMD^2 values than LLMSYN-full across all code pairs, indicating better alignment with real joint distributions. MAIN+PROC and MAIN+COMORB pairs show modest divergence (≈ 0.16–0.18), while COMORB+PROC displays the largest gap, where LLMSYN-full has substantially higher MMD (0.1842) compared to base and prior (≈ 0.018). Overall, synthetic data struggles most with comorbidity-related relationships, while base/prior variants preserve joint patterns more faithfully than the full model.

Overall, LLMSYN-full and LLMSYN-prior consistently reproduce demographic distributions, while fidelity for diagnosis and comorbidity patterns remains low across all models.

6.3 Additional Extensions and Ablations

Using GPT 5.1 (Thinking), we explored several extensions, and the most feasible was an ablation that removed all comorbidities from synthetic records (`OTHER_ICD_COUNT`=0). We regenerated a 100-patient synthetic cohort under this constraint and reran all utility and fidelity evaluations. Utility showed no meaningful improvement: respiratory AUROC remained 0.45–0.56 (REAL baseline

Table 4: MMD^2 between synthetic and real joint distributions (lower = better).

Pair	Full	Prior	Base
MAIN + PROCEDURE	0.0385	0.0260	**0.0259**
MAIN + COMORBIDITY	0.1770	0.1601	**0.1596**
COMORB + PROCEDURE	0.1842	**0.0184**	**0.0181**

0.562) and mortality AUROC 0.43–0.58 (REAL 0.584), with F1 scores near zero due to imbalance. Fidelity changed only marginally: comorbidity count matched perfectly (KS=0), MAIN+PROC MMD stayed similar ($\approx$ 0.05–0.07), MAIN+COMORB remained high ($\approx$ 0.53–0.58), and COMORB+PROC worsened for `syn_full`.

Overall, this ablation confirms the LLMSYN finding that, at 100-sample scale, comorbidities add distributional richness but have limited impact on predictive utility, and diagnosis–procedure mismatch remains the primary fidelity bottleneck. Prompt-engineering constraints (e.g., asking for feasible extensions) produced more actionable suggestions.

7 Discussion

LLMSYN is partially reproducible. We successfully recreated the four-step pipeline and key qualitative trends, data augmentation preserves AUROC relative to real-only training, fully synthetic data performs worse but remains usable, and demographic fidelity is strong while diagnosis/comorbidity fidelity is limited. However, numerical differences persist because our cohort is much smaller (100 samples), class imbalance is stronger, and we used different LLMs, priors, and grounding sources.

Reproduction was generally straightforward given the prompt-driven nature of the method, and LLMs simplified preprocessing and metric implementation. The main challenges were missing original prompts, Mayo grounding text, and ICD/procedure prior tables, which required reconstructing templates manually; fidelity metrics were also unstable at this scale. To improve reproducibility, we recommend releasing full prompt templates, grounding snippets, example synthetic JSON, and explicit priors. Overall, LLMSYN's methodology and qualitative trends are reproducible, but exact numerical replication is limited by dataset scale and unavailable knowledge sources.

8 Author Contributions

Sahil Singhal designed and executed all LLM prompt templates for demographics, diagnosis, comorbidities, and procedure generation, validated synthetic outputs, and supported preprocessing checks, variant testing, and report writing. **Goyeun Yun** implemented the preprocessing pipeline, constructed all priors, designed and ran all evaluation experiments (utility, fidelity, ablations), generated analyses and visualizations, and wrote the majority of the report.

Both authors jointly reviewed experimental design decisions, refined prompt strategies, and contributed to the overall structure and interpretation of results.

Appendix A: Construction of `prior.json`

This appendix summarizes how population-level priors were constructed for LLMSYN. All priors use only aggregated MIMIC-III (v1.4) statistics and contain no patient-level data.

A.1 Data Sources and Cohort Definition

We follow the LLMSYN cohort logic using the following MIMIC-III tables: `ADMISSIONS`, `PATIENTS`, `DIAGNOSES_ICD`, `D_ICD_DIAGNOSES`.

Cohort filtering: (1) Remove all `NEWBORN` admissions. (2) For each subject, select the first adult admission for demographics. (3) Mortality is taken from the final admission.

Age is computed as AGE $=$ years(ADMITTIME $-$ DOB), clipped to $[0, 120]$ and binned into `0{17`, `18{34`, `35{49`, `50{64`, `65{79`, `80+`.

A.2 Contents of `prior.json`

The constructed file stores:

- `mortality_rate`: **0.1478**
- Demographic distributions: LANGUAGE, RELIGION, MARITAL_STATUS, ETHNICITY, INSURANCE, GENDER, AGE_BIN
- `top100_icd9_path`: `outputs/priors/top100_icd9.csv`
- `cohort`: "`no_newborn_filter_first_admission`"

A.3 Example Prior Distributions

Table 5: Selected demographic priors from `prior.json`

Category	Top Entries (Proportion)
LANGUAGE	ENGL (0.556), None (0.358), SPAN (0.020)
RELIGION	Catholic (0.351), Not Specified (0.211), Unobtainable (0.131)
MARITAL	Married (0.477), Single (0.243), Widowed (0.139)
ETHNICITY	White (0.705), Unknown (0.098), Black (0.070)
INSURANCE	Medicare (0.525), Private (0.347), Medicaid (0.083)
GENDER	Male (0.566), Female (0.434)
AGE_BIN	65–79 (0.306), 50–64 (0.271), 80+ (0.151)

A.4 ICD-9 Frequency Priors

ICD-9 frequencies are computed from all non-newborn admissions:

- Count codes in `DIAGNOSES_ICD`
- Sort by descending frequency
- Keep the top 100 codes
- Save as `top100_icd9_non_newborn.csv`

Appendix B: Dataset Statistical Summary

This section provides descriptive statistics for all datasets used in our reproducibility study: (1) real MIMIC-III subsample, (2) LLMSYN_full, (3) LLMSYN_prior, and (4) LLMSYN_base. All synthetic datasets contain 100 generated patients.

Table 6: Demographic Statistics

Dataset	**N**	**AGE (mean±std)**	**Min–Max**	**Mortality (%)**
REAL (MIMIC)	36,557	62.36±16.80	18–89	10.6
LLMSYN_full	100	59.41±20.99	3–95	15.0
LLMSYN_prior	100	60.44±21.54	1–95	18.0
LLMSYN_base	100	46.52±26.33	3–94	15.0

Table 7: Missingness and EHR Structure Statistics

Dataset	**Lang Unk (%)**	**Rel Unk (%)**	**Marital Unk (%)**	**Comorb / Proc (mean/med)**
REAL (MIMIC)	35.6	1.1	5.7	10.05/8 / 5.80/4
LLMSYN_full	36.0	0.0	6.0	4.22/4 / 4.51/5
LLMSYN_prior	41.0	0.0	4.0	1.99/2 / 3.05/3
LLMSYN_base	27.0	0.0	15.0	2.99/3 / 2.79/3

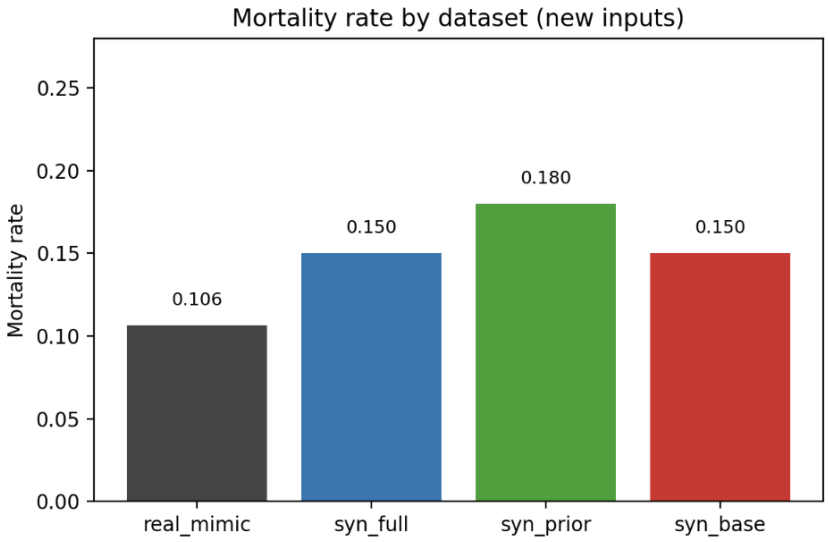

Figure 2: Mortality rate

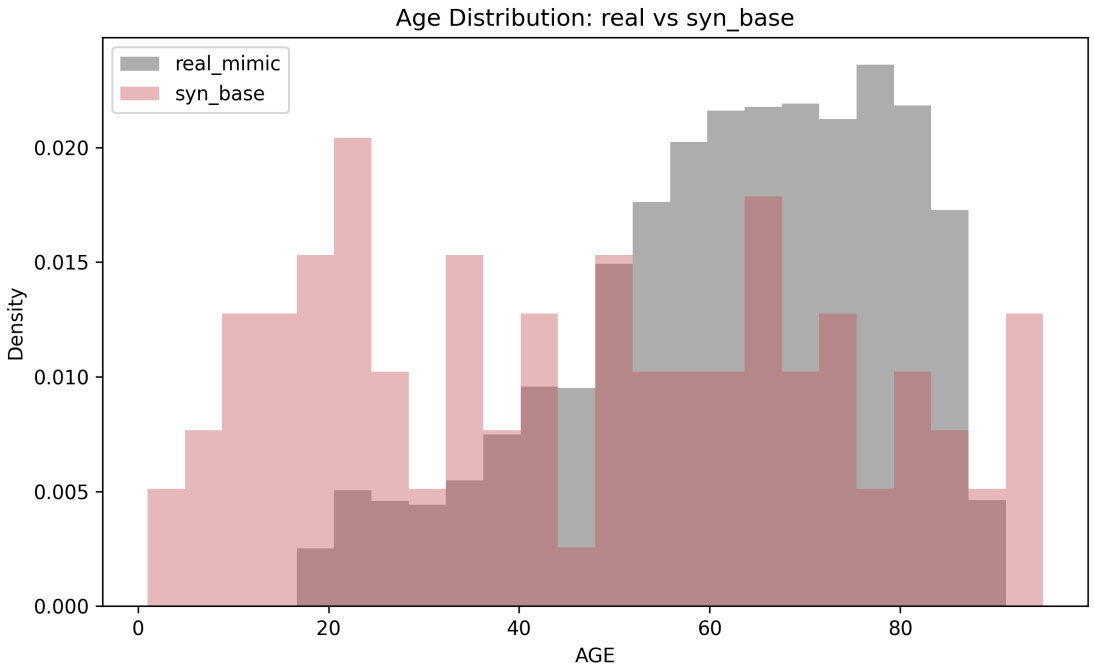

Figure 3: Age Distribution (LLMSYN_base)

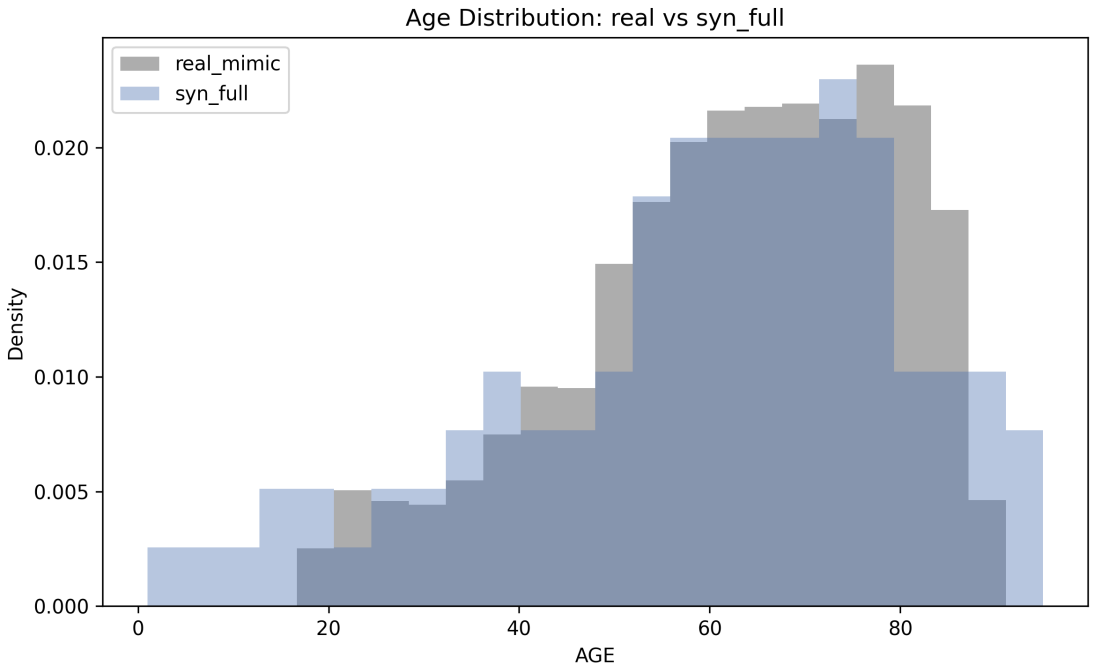

Figure 4: Age Distribution (LLMSYN_full)

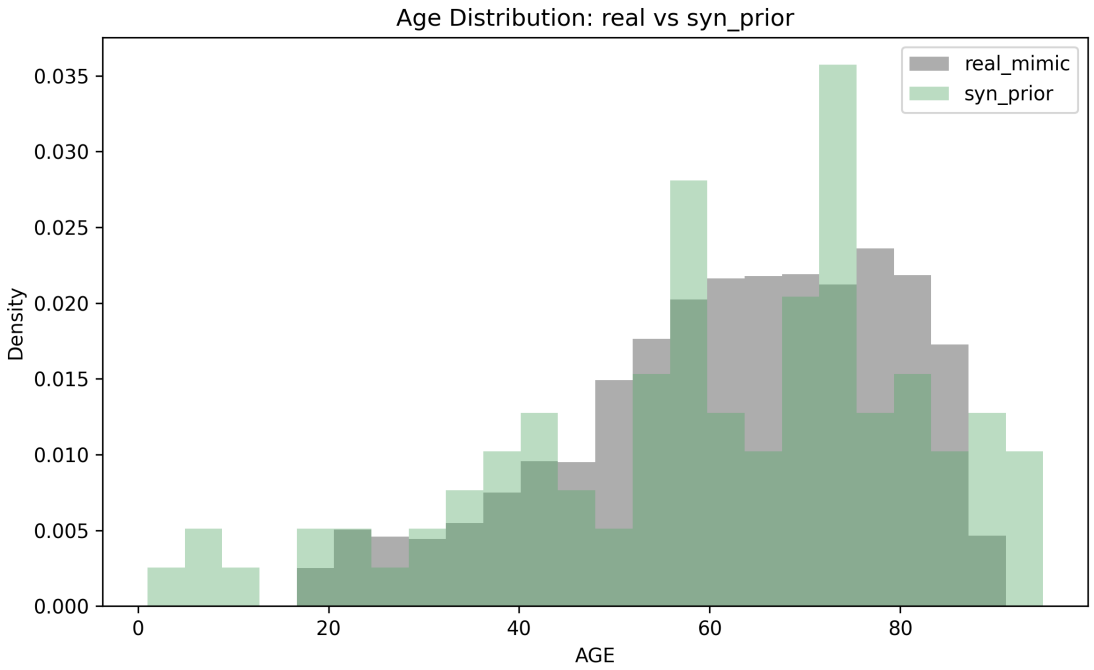

Figure 5: Age Distribution (LLMSYN_prior)

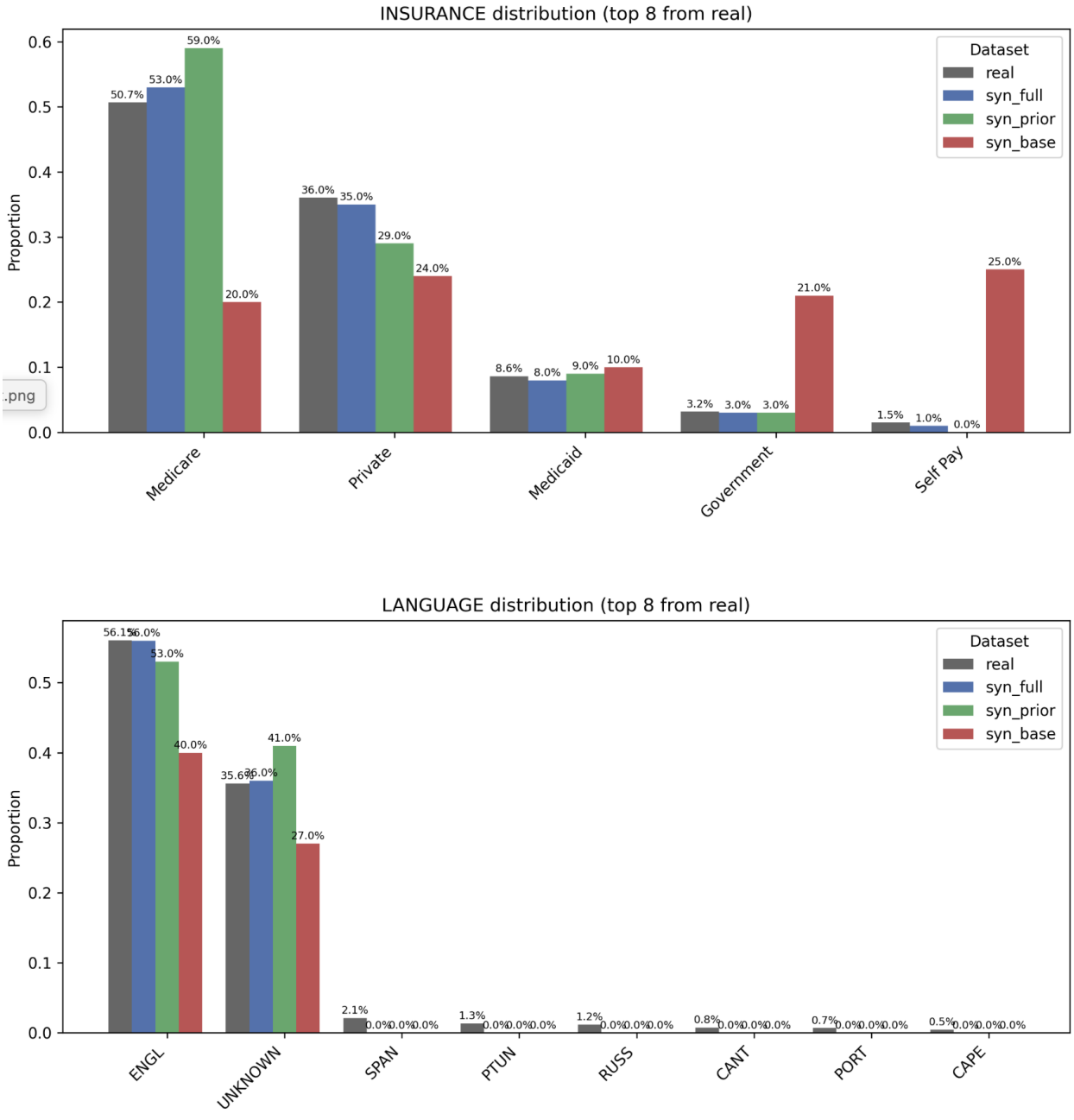

Figure 6: Insurance (top) and Language (bottom) Distributions

Appendix C: LLM Prompt Logs and Generated Code

C.1 Step 1: Demographics Generation (x0)

C.1.1 Prompt

```
Task: You are a hospital intake expert responsible for generating
 ↪  realistic
patient demographic records for a synthetic EHR dataset. Use
 ↪  reasoning and
public clinical knowledge only. Do not use any real patient data.

Retrieval / Prior Knowledge:
[mortality_rate = 0.14778]
[LANGUAGE, RELIGION, MARITAL_STATUS, ETHNICITY, INSURANCE,
 ↪  GENDER, AGE_BIN priors]

Description:
Generate AGE, LANGUAGE, RELIGION, MARITAL STATUS, ETHNICITY,
 ↪  INSURANCE,
HOSPITAL EXPIRE FLAG for exactly 100 patients.

Steps:
1. Follow priors proportionally.
2. Ages 1{17 cannot be MARRIED/WIDOWED/DIVORCED.
3. Output a clean 100-row dataframe only.
```

C.1.2 Validation

- Age-bin proportions roughly matched priors.
- Some minors were incorrectly labeled as MARRIED.
- Insurance distribution deviated from priors.
- Mortality rate was too high (around 0.25 vs. target 0.14778).

C.1.3 Error Summary

The LLM partially ignored minor-age constraints and insurance/mortality priors.

C.1.4 Revised Prompt

```
Regenerate the demographics strictly enforcing:
- No minors in MARRIED/WIDOWED/DIVORCED.
- Insurance must follow the given priors.
- Mortality must follow Bernoulli(0.14778).
- Output a clean CSV with exactly 100 rows and no explanations.
```

C.2 Step 2: Primary Diagnosis Generation (x1)

C.2.1 Prompt

```
You are a professional medical coder assigning ICD-9 primary
 ↪  diagnoses.
```

```
Rules:
1. Use only mayoclinic.org for reasoning.
2. Assign MAIN_DIAGNOSIS using only ICD-9 Top 100 list.
3. No hallucinated codes; no codes outside the list.
4. Add a MAIN_DIAGNOSIS column to the existing demographics
 ↪  table.

Input:
{patient demographics}

ICD9 Top 100 List:
{icd9_top100_string}
```

C.2.2 Validation

- Most outputs used valid ICD-9 codes from the list.
- Decimal formatting was inconsistent or missing.
- Some age-inappropriate diagnoses were assigned (e.g., Alzheimer's for young adults).

C.2.3 Error Summary

The LLM ignored ICD-9 decimal formatting and age-appropriate diagnosis constraints.

C.2.4 Revised Prompt

```
Ensure correct ICD-9 decimal formatting.
Avoid diagnoses that are inappropriate for patient age < 50
 ↪  (e.g., dementia).
Always select codes directly from the Top 100 ICD-9 list.
Return only the updated dataframe with a MAIN_DIAGNOSIS column.
```

C.3 Step 3: Comorbidity Generation (x2)

C.3.1 Prompt

```
Assign 0{6 ICD-9 comorbidities for each patient.

Rules:
1. Use mayoClinic.org for clinical reasoning.
2. Use only codes from the ICD-9 Top 100 list.
3. No duplicate comorbidities for the same patient.
4. Comorbidities must be clinically plausible given the main
 ↪  diagnosis and age.

Input:
{patient + MAIN_DIAGNOSIS}

ICD9 Top 100 List:
{icd9_top100_string}
```

C.3.2 Validation

- Many comorbidities were clinically relevant.
- Some patients had duplicated ICD-9 codes in the list.
- A few hallucinated codes appeared (e.g., non-standard suffixes like "401.1A").

C.3.3 Error Summary

The LLM generated invalid ICD-9 variants and allowed duplicates within a patient.

C.3.4 Revised Prompt

```
Regenerate the COMORBIDITIES column ensuring:
- No duplicate ICD-9 codes per patient.
- No hallucinated or partially-matched codes.
- Each code must match EXACTLY one entry in the ICD-9 Top 100
 ↪  list.
```

C.4 Step 4: Procedure Assignment (x3)

C.4.1 Prompt

```
Assign ICD-9 procedure codes using demographics, main diagnosis,
 ↪  and comorbidities.

Rules:
1. Use mayoClinic.org for clinical reasoning.
2. Use only codes from the ICD-9 Procedure Master List.
3. Assign 0{6 procedures per patient.
4. Ensure procedures are clinically aligned with the main
 ↪  diagnosis and comorbidities.

Input:
{patient + comorbidities}

Procedure Master List:
{icd9_proc_master_string}
```

C.4.2 Validation

- Several procedures were appropriate for the main diagnosis.
- Some outputs contained modern CPT-like or non-ICD9 codes.
- A few diagnosis–procedure mismatches remained.

C.4.3 Error Summary

The LLM sometimes used modern procedure codes outside the ICD-9 list and ignored diagnosis–procedure consistency.

C.4.4 Revised Prompt

```
Use ONLY valid ICD-9 procedure codes from the given master list.
Do NOT use CPT or modern codes.
```

```
Ensure that each procedure is clinically consistent with the main
 ↪  diagnosis
and comorbidities. Output only the updated dataframe with
 ↪  PROCEDURE codes.
```

C.5 LLM-Assisted Preprocessing Code

C.5.1 Initial Prompt

```
You are a data engineer helping me write preprocessing code for
 ↪  LLMSYN.
I need Python code that:
1) Loads the MIMIC-III derived tables (demo.csv, diagnoses.csv,
 ↪  cpt.csv)
2) Keeps only the first non-newborn admission per patient
3) Computes ICD9 counts, CPT counts, OTHER_ICD counts
4) Merges them into a single tabular dataset for downstream
 ↪  models
5) Outputs a clean CSV

Return only code with comments. Do not include explanations.
```

C.5.2 Initial LLM Output (Summary)

The LLM produced skeleton code that:

- loaded the three CSVs,
- merged them only on `SUBJECT_ID`,
- computed simple `ICD9_COUNT` and `CPT_COUNT` per patient,
- and wrote a single `processed.csv`.

C.5.3 Validation

- **Correctness:** Basic merging and counting were correct, but:
  - NEWBORN admissions were not removed.
  - There was no logic to keep only the first non-newborn admission.
  - `OTHER_ICD_COUNT` was not computed.
  - Admission-level joins via `HADM_ID` were missing.
- **Relevance:** The flow resembled generic EHR preprocessing, not the LLMSYN cohort rules.
- **Helpfulness:** The code served as a useful starting point but required substantial modification.

C.5.4 What Was Wrong With the Initial Prompt?

- It did not explicitly mention NEWBORN exclusion and "first non-newborn admission" logic.
- It did not specify that joins must be on both `SUBJECT_ID` and `HADM_ID`.
- LLMSYN-specific cohort filtering conditions were omitted, leading to a generic solution.

C.5.5 Prompt Count (Approximate)

Initial prompt: 1; follow-up corrections: 4; debug/formatting: 2. **Total:** approximately 7 prompts.

C.6 LLM-Assisted Feature Engineering

C.6.1 Initial Prompt

```
I need Python code that creates a binary respiratory failure
 ↪  label from ICD-9
codes in an LLMSYN-style CSV file. The script should:

1) Load the flat CSV (REAL or SYN)
2) Normalize ICD9 codes (remove dots, handle strings)
3) Identify respiratory-related ICD9 codes (460519) and create a
 ↪  label
4) Build simple feature arrays (X and y)
5) Return only Python code.
```

C.6.2 Initial LLM Output (Summary)

The LLM returned a minimal snippet that:

- read a CSV file,
- extracted the first three characters of the ICD9 string,
- assigned label = 1 if the code was in [460, 519],
- set X to all remaining columns after dropping the label.

C.6.3 Validation

- **Correctness:** The respiratory range was conceptually correct, but:
  - ICD9 normalization (dots, variable lengths) was not handled.
  - Non-numeric ICD codes caused errors.
- **Relevance:** The code did not include demographic or comorbidity features required by our model.
- **Helpfulness:** It was a minimal starting point but far from the final feature engineering pipeline.

C.6.4 What Was Wrong With the Initial Prompt?

- It did not mention:
  - COMORBIDITIES parsing from list-like strings,
  - categorical encoding of LANGUAGE, ETHNICITY, INSURANCE, RELIGION, MARITAL_STATUS,
  - generation of `OTHER_ICD_COUNT`,
  - saving `X.npy`, `y.npy`, and feature names.

C.6.5 Prompt Count (Approximate)

Initial: 1; feature additions and parsing fixes: 4; debugging and shaping: 3. **Total:** approximately 8 prompts.

C.7 LLM-Assisted Training Code Generation

C.7.1 Context

The LLMSYN reproduction does not train a generative model; instead, we train supervised Random Forest models for respiratory failure and mortality prediction to assess synthetic data utility. The "training loop" here refers to these supervised experiments.

C.7.2 Initial Prompt

```
I need Python code that trains a Random Forest classifier for my
 ↪  LLMSYN
reproduction experiments. The code should:

1) Load REAL and SYN datasets
2) Split the REAL dataset into train/test
3) Train a model under FS, REAL, and DA settings
4) Evaluate using accuracy, AUROC, and F1
5) Use a fixed random seed

Please generate the full training loop in Python.
```

C.7.3 Initial LLM Output (Summary)

The initial output:

- trained a Random Forest on a single dataset,
- evaluated on the same data (no separate test set),
- did not implement FS/REAL/DA variants,
- did not separate respiratory failure vs. mortality tasks,
- omitted any fixed `random_state`.

C.7.4 Validation

- **Correctness:** Basic Random Forest usage was correct but led to data leakage.
- **Relevance:** The code did not reflect LLMSYN-style experimental setups.
- **Helpfulness:** Useful as a template, but required substantial restructuring.

C.7.5 What Was Wrong With the Initial Prompt?

- It did not clearly specify:
  - that evaluation must always be on the REAL test set,
  - stratified splitting on the target label,
  - separate handling of respiratory failure vs. mortality,
  - detailed definitions of FS, REAL, and DA settings.

C.7.6 Prompt Count (Approximate)

Initial: 1; design of FS/REAL/DA experiment function: 4; debugging (splits, metrics): 3. **Total:** approximately 8 prompts.

C.8 LLM-Assisted Evaluation Metrics Implementation

C.8.1 Initial Prompt

```
I need Python code to evaluate synthetic EHR data for my LLMSYN
 ↪  reproduction.
Please generate code that computes:

1) Utility metrics:
   - Accuracy
   - AUROC
   - F1 score

2) Fidelity metrics:
   - Kolmogorov{Smirnov (KS) statistic for multiple features
   - Maximum Mean Discrepancy (MMD) between synthetic and real
    ↪  data

3) Privacy metrics:
   - k-anonymity violation counts

Return only Python code.
```

C.8.2 Initial LLM Output (Summary)

The LLM produced:

- a Random Forest evaluation snippet computing ACC and F1,
- an example of KS statistic computation using `ks_2samp`,
- no implementation of MMD,
- no k-anonymity checks,
- no distinction between FS, REAL, and DA settings.

C.8.3 Validation

- **Correctness:** ACC and F1 implementations were correct; KS was technically valid.
- **Relevance:** Only partially covered the full evaluation protocol from LLMSYN.
- **Helpfulness:** Served as a starting point, but substantial manual refinement was required for MMD and privacy metrics.

C.8.4 What Was Wrong With the Initial Prompt?

- It did not specify:
  - which features to use for KS tests,
  - the exact MMD formulation (kernel choice, bandwidth),
  - how to group records for k-anonymity,
  - how to evaluate each experimental setting (FS, REAL, DA) separately.

C.8.5 Prompt Count (Approximate)

Initial: 1; RF utility/evaluation fixes: 3; KS/fidelity refinement: 3; MMD implementation: 3–4; k-anonymity: 2. **Estimated total:** roughly 10–13 prompts.

References

[1] Mayo clinic: Diseases and conditions. https://www.mayoclinic.org/diseases-conditions. Accessed: 2025-11-23.

[2] Synthetic patient generator (synthea). https://synthea.mitre.org/, 2020.

[3] Nicholas Carlini, Florian Tramer, Eric Wallace, et al. Extracting training data from large language models. In *USENIX Security Symposium*, pages 2633–2650, 2021.

[4] Centers for Medicare & Medicaid Services. Healthcare common procedure coding system (hcpcs). https://www.cms.gov/Medicare/Coding/MedHCPCSGenInfo. Accessed: 2025-11-23.

[5] Centers for Medicare & Medicaid Services. Icd-9-cm diagnosis codes. https://www.cms.gov/medicare/icd-9-cm-diagnosis-codes. Accessed: 2025-11-23.

[6] Matthew Fredrikson, Eric Lantz, Somesh Jha, et al. Privacy in pharmacogenetics: An end-to-end case study. In *USENIX Security Symposium*, pages 17–32, 2014.

[7] Arthur Gretton, Karsten M Borgwardt, Malte J Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. In *Journal of Machine Learning Research*, volume 13, pages 723–773, 2012.

[8] Yijie Hao, Joyce C Ho, and Huan He. Llmsyn: Generating synthetic electronic health records without patient-level data. *Proceedings of Machine Learning Research*, 252:1–27, 2024.

[9] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

[10] Alistair EW Johnson, Tom J Pollard, Lu Shen, et al. Mimic-iii, a freely accessible critical care database. *Scientific Data*, 3:160035, 2016.

[11] Yikuan Li, Shishir Rao, José Roberto Ayala Solares, et al. Behrt: Transformer for electronic health records. *Scientific Reports*, 10:7155, 2020.

[12] Wes McKinney. Data structures for statistical computing in python. pages 51–56, 2010.

[13] Arvind Narayanan and Vitaly Shmatikov. Robust de-anonymization of large sparse datasets. In *IEEE Symposium on Security and Privacy*, pages 111–125, 2008.

[14] Fabian Pedregosa et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[15] Laila Rasmy, Yang Xiang, Ziqian Xie, Cui Tao, and Degui Zhi. Med-bert: pretrained contextualized embeddings on large-scale structured electronic health records. *NPJ Digital Medicine*, 4:86, 2021.

[16] SciPy Developers. scipy.stats.ks_2samp. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html. Accessed: 2025-11-23.

[17] Ethan Steinberg, Ken Jung, Jason A Fries, Conor K Corbin, Stephen R Pfohl, and Nigam H Shah. Language models are an effective patient representation learning technique. 2020.